

Apuntes Semana 13 Clase I - Inteligencia Artificial

Clase del 19 de Mayo de 2026

Alexander Brenes Garita

Carnet: 2018191805

I. RETROALIMENTACIÓN DEL PROYECTO I

A. Aspectos Técnicos y de Investigación

El profesor destacó que, si bien los proyectos entregados presentaron resultados generalmente buenos, se identificaron deficiencias importantes en la profundidad técnica e investigativa. El principal problema detectado fue el uso excesivo de herramientas de IA generativa (Claude, ChatGPT, etc.) para redactar el informe y generar código, sin que los estudiantes dominaran los conceptos subyacentes.

“Si alguien experto en el campo lee su informe y nota que un concepto está explicado incorrectamente, su trabajo se ve afectado. El conocimiento profundo es el factor diferenciador en el mercado laboral.”

Se enfatizó que el ingeniero debe ser quien tome las decisiones y quien valide cada línea de código generado, no quien simplemente lo acepta sin comprender.

B. Deficiencias en los Informes

Se identificaron los siguientes problemas recurrentes en los informes:

- **Falta de profundidad:** Conceptos como la Transformada Rápida de Fourier (FFT) fueron mencionados sin explicar su funcionamiento o justificación matemática.
- **Exceso de páginas:** Varios informes superaron las 20 páginas, cuando el estándar en conferencias científicas es de 6 a 8 páginas incluyendo referencias.
- **Citas incompletas:** Se rellenó la sección de referencias sin incluir citas en el cuerpo del texto.
- **Introducción débil:** Muchos trabajos carecieron de un planteamiento claro del problema, revisión del estado del arte, y descripción de la metodología seguida.
- **Subutilización de Weights & Biases:** La herramienta fue usada de forma superficial, sin aprovechar dashboards de curvas de pérdida, análisis de overfitting, o visualización de imágenes por época.

C. Cambios para Próximos Proyectos

- Los informes **no deben superar las 10 páginas**.
- Las rúbricas serán más estrictas en cuanto a calidad técnica.
- Es obligatorio incluir: estado del arte con citas académicas, análisis de overfitting en WandB, y gráficas de pérdida de entrenamiento y validación.

II. AUTOENCODERS: REVISIÓN TÉCNICA

A. Concepto General

Un autoencoder es un algoritmo de aprendizaje **auto-supervisado** (no supervisado). Aprende a comprimir y reconstruir sus propios datos de entrada, utilizando la misma muestra tanto como entrada como como etiqueta implícita. La arquitectura base está compuesta por tres componentes:

- 1) **Encoder:** Red de convoluciones que reduce las dimensiones de la imagen, extrayendo características relevantes.
- 2) **Cuello de botella / Vector latente:** Representación comprimida en un espacio vectorial de menor dimensión.
- 3) **Decoder:** Proceso inverso de *upsampling* que reconstruye la imagen a partir del vector latente.

La función de pérdida compara directamente la imagen reconstruida con la imagen original (píxel a píxel o mediante métricas de similitud estructural), lo que constituye el elemento supervisado implícito del algoritmo.

B. Tipos de Autoencoders

1) **Autoencoder Vanilla (Clásico):** El autoencoder estándar comprime la información en un cuello de botella y la reconstruye. Cuanto más pequeño sea el vector latente, mayor es la compresión pero menor la calidad de reconstrucción.

2) **Autoencoder Disperso (Sparse):** Mantiene la misma dimensionalidad en la arquitectura, pero aplica un mecanismo de encendido/apagado aleatorio de neuronas (similar a *dropout*) para distribuir la información de forma más eficiente entre las neuronas.

3) **Autoencoder Denoising:** Diseñado para remover ruido de imágenes. El proceso de entrenamiento consiste en:

- 1) Tomar la imagen limpia original.
- 2) Agregarle ruido artificialmente (ej. *salt & pepper*).
- 3) Alimentar la imagen ruidosa al autoencoder.
- 4) Comparar la salida con la **imagen limpia original** (no con la ruidosa).

Esto permite entrenar modelos que transforman imágenes de baja resolución a alta resolución, de escala de grises a color, o que eliminan tipos específicos de artefactos visuales.

4) **Autoencoder Variacional (VAE):** El VAE introduce una mejora fundamental: en lugar de un vector latente determinístico, el encoder produce una **distribución de probabilidad** caracterizada por su media (μ) y desviación estándar (σ) para cada dimensión del espacio latente.

Esto permite tener un **espacio latente continuo**, con el cual se puede interpolar entre clases y generar imágenes que el modelo nunca ha visto durante el entrenamiento. El proceso de entrenamiento utiliza el *truco de reparametrización* para poder hacer backpropagation a través de la muestra estocástica.

La función de pérdida del VAE tiene dos componentes:

$$\mathcal{L}_{VAE} = \mathcal{L}_{reconstrucción} + \mathcal{L}_{KL} \quad (1)$$

donde $\mathcal{L}_{reconstrucción}$ (BCE o MSE) penaliza la diferencia entre la imagen reconstruida y la original, y $\mathcal{L}_{KL}$ (divergencia KL) penaliza que la distribución aprendida se aleje de una distribución normal estándar $\mathcal{N}(0, 1)$.

“Si logramos que las características se distribuyan en un espacio latente continuo, podemos cambiar los valores y ver cómo afectan el resultado: cambiar el género, el tono de piel, la sonrisa... generando imágenes que el modelo jamás fue entrenado para ver.”

5) *U-Net con Skip Connections*: La arquitectura U-Net fue propuesta originalmente para segmentación de imágenes biomédicas. Su diferencia principal respecto al autoencoder clásico es el uso de **skip connections**: conexiones directas entre las capas del encoder y las capas correspondientes del decoder.

En lugar de la suma elemento a elemento (como en ResNet), U-Net usa **concatenación** de los tensores, lo que permite al decoder acceder a información de alta resolución preservada del encoder. Esto resulta en reconstrucciones con mayor fidelidad y detalle.

Aplicaciones: segmentación de imágenes médicas (rayos X, detección de cáncer), segmentación de terrenos satelitales, cualquier tarea donde se requiera generar una máscara o salida de la misma resolución que la entrada.

C. Aplicaciones de los Autoencoders

- **Reducción de dimensionalidad**: Superiores a PCA en tareas con imágenes. Permiten comparaciones semánticas de similitud entre imágenes mediante distancia vectorial en el espacio latente.
- **Detección de anomalías**: Un autoencoder entrenado solo con muestras normales producirá vectores latentes erróneos y reconstrucciones de baja calidad ante muestras anómalas (alta función de pérdida = anomalía).
- **Denoising**: Remoción de ruido, mejora de resolución, conversión de dominio.
- **Generación de imágenes**: Con VAE, interpolación entre clases y generación de muestras nuevas.
- **Segmentación**: Con U-Net, generación de máscaras de segmentación.

D. Hiperparámetros Clave

- **Tamaño del vector latente**: Más grande $\rightarrow$ mayor calidad, más memoria. Más pequeño $\rightarrow$ mayor compresión.
- **Profundidad de la red**: A mayor profundidad, mayor capacidad pero más costo computacional y riesgo de sobreajuste.

- **Arquitectura del encoder**: VGG, ResNet, U-Net, convoluciones simples, etc.
- **Función de pérdida**: L1 (MAE), L2 (MSE), SSIM, MS-SSIM. L2 tiende a producir imágenes más borrosas (*blur*); métricas estructurales como SSIM capturan mejor la percepción humana.

III. TAREA 2: DETECCIÓN DE ANOMALÍAS CON AUTOENCODERS

A. Objetivo

Implementar y comparar dos arquitecturas de autoencoders para detección de anomalías en un dataset industrial:

- Autoencoder Variacional (VAE)
- Autoencoder con arquitectura U-Net (skip connections)

B. Dataset: MVTec Anomaly Detection

Dataset de imágenes industriales con objetos que presentan tanto muestras normales como defectuosas. Para esta tarea se utilizarán **cuatro clases**:

- Cables
- Cápsulas
- Tornillos
- Transistores

Tamaño de imágenes: $128 \times 128 \times 3$ (color).

C. Herramientas Requeridas

1) *PyTorch Lightning*: Framework que añade una capa de abstracción sobre PyTorch, organizando el código de entrenamiento en métodos bien definidos (`training_step`, `validation_step`, etc.). Permite gestionar el ciclo de entrenamiento de forma modular, con soporte nativo para *callbacks*, *profiling* y registro de métricas.

2) *Hydra*: Sistema de gestión de configuraciones mediante archivos YAML. Permite separar los hiperparámetros del código, facilitar la ejecución de múltiples experimentos y reproducir configuraciones exactas. La estructura mínima del proyecto debe incluir:

- `configs/config.yaml` (archivo central)
- `configs/model/` (arquitectura, tamaño latente, capas)
- `configs/trainer/` (épocas, learning rate, batch size)
- `configs/logger/` (configuración de WandB)

3) *Weights & Biases (WandB)*: Plataforma de seguimiento de experimentos. Se debe registrar:

- Curvas de pérdida (entrenamiento y validación)
- Reconstrucciones de imágenes por época
- Proyección del espacio latente (t-SNE en 2D)
- Histogramas del error de reconstrucción

D. Configuración Experimental

Se deben realizar **8 entrenamientos** (4 por arquitectura) variando la función de pérdida:

- 1) Distancia L1 (MAE)
- 2) Distancia L2 (MSE)
- 3) SSIM (*Structural Similarity Index*)
- 4) MS-SSIM (*Multi-Scale SSIM*)

E. Evaluación

- **Reconstrucción:** Comparar imágenes normales vs. reconstruidas; evaluar la calidad de reconstrucción en imágenes con defectos.
- **Proyección del espacio latente:** Visualizar con t-SNE cómo se distribuyen las clases normales y anómalas.
- **Histograma del error:** Analizar si las imágenes defectuosas producen errores de reconstrucción significativamente mayores.

F. Formato de Entrega

No se solicita informe escrito. La entrega consiste en un **Jupyter Notebook** con:

- Celdas de texto explicativas (Markdown) documentando cada paso.
- Imágenes generadas durante el entrenamiento integradas en el notebook.
- Todas las visualizaciones de WandB incorporadas.

IV. PRÓXIMAS ACTIVIDADES

A. Lecturas Obligatorias para la Próxima Clase

La siguiente semana se hará una pausa en el tema de imágenes para introducir **Procesamiento de Lenguaje Natural (NLP)**. Se deben leer dos papers:

- 1) **Paper sobre modelos de lenguaje tipo PAY:** Cubre la arquitectura base de transformers y sus capacidades de representación del lenguaje. Se evaluará la comprensión matemática del modelo.
- 2) **Crítica a los modelos de lenguaje:** Paper publicado hace aproximadamente cinco años que cuestiona el comportamiento de los LLMs como “loros estocásticos” que repiten patrones del internet sin comprensión real. Aunque el campo ha evolucionado, sus principios siguen siendo válidos y sentaron precedentes importantes.

B. Calendario de Evaluaciones

TABLE I
PRÓXIMAS EVALUACIONES DEL CURSO

Actividad	Plazo	Peso
Quiz (papers NLP)	Próximo martes	Normal
Tarea 2 (Autoencoders)	Próxima semana	×2
Proyecto 2	Semana 15	Proyecto
Examen Final	Semana 18	Examen

“El examen es en la semana 18. Ya les subí las notas al sistema digital. Próximo martes hay quiz sobre los papers de NLP — tengan eso muy presente.”

V. CONCEPTOS CLAVE PARA EL QUIZ

Basado en el contenido de clase, los siguientes conceptos son prioritarios para el quiz y el examen:

- Diferencia entre autoencoder vanilla, disperso, denoising y variacional.
- Por qué el autoencoder es considerado auto-supervisado a pesar de usar descenso del gradiente.

- Función de pérdida del VAE: componente de reconstrucción + divergencia KL.
- Truco de reparametrización en el VAE.
- Por qué U-Net usa concatenación y no suma en sus skip connections.
- Cómo el espacio latente continuo del VAE permite generar nuevas imágenes.
- Principio de detección de anomalías con autoencoders.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*. MIT Press, 2016.
- [2] D. P. Kingma, M. Welling, “Auto-Encoding Variational Bayes,” *arXiv:1312.6114*, 2013.
- [3] O. Ronneberger, P. Fischer, T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *MICCAI*, 2015.
- [4] P. Bergmann et al., “MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection,” *CVPR*, 2019.
- [5] E. M. Bender, T. Gebru, A. McMillan-Major, S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?,” *FAccT*, 2021.